UNIVERSITY OF KELANIYA — FACULTY OF COMPUTING AND TECHNOLOGY

Department of Software Engineering | SENG 34213: System Development Project

Academic Year 2025/2026

ACADEMIC & TECHNICAL RELEASE EVALUATION

Performance Test Results Report

Darshana Opticals — Optical Shop Management System (OSMS)

Deliverable Reference:	documents/testing/performance-report.pdf
GitHub Issue Reference:	[DDP-086] Issue #92 (Performance & Reliability)
Document Version:	Version 1.0 (Official Release Gate)
Evaluation Date:	October 2026
Project Domain:	Enterprise Retail, Clinical & Inventory Management
Release Assessment:	**CONDITIONALLY APPROVED (Local Benchmarks Met)**

Prepared in accordance with SENG 34213 System Development Project Guidelines (Chapters 5, 6, 7, 8, 10 & Appendix A).

© 2026 Darshana Opticals (Pvt) Ltd. All Rights Reserved. Confidential Academic Assessment.

Table of Contents

Sec	Report Section	Status
1	Cover Page & Document Metadata	Completed
2	Executive Summary	Completed
3	Introduction & System Background	Completed
4	Testing Objectives & Quality Targets	Completed
5	Scope of Testing & Boundaries	Completed
6	Test Environment & Safety Harness Architecture	Completed
7	Test Methodology & Workload Modeling	Completed
8	Empirical Performance Test Results	Completed
9	Charts and Visualizations	Completed
10	Bottleneck Profiling & Index Optimizations	Completed
11	Reliability, Concurrency & Fault Recovery Results	Completed
12	Performance Requirement Evaluation (SRS Non-Functional Alignment)	Completed
13	Engineering Recommendations	Completed
14	Environmental Limitations & Test Boundaries	Completed
15	Conclusion & Release Gate Sign-Off	Completed
16	References, Git Evidence & Reproducibility Guide	Completed

2. Executive Summary

This report documents the formal empirical performance, concurrency, query efficiency, and reliability evaluation conducted for the **Darshana Opticals Optical Shop Management System (OSMS) Backend** under GitHub Issue **#92 (DDP-086)**. Testing was executed to validate compliance with the non-functional requirements (NFRs) specified in the approved *Software Requirements Specification (SRS v1.0)*, verify transactional consistency under race conditions, evaluate database execution plan efficiency, and establish whether the system satisfies release gate criteria for Sprint 2/3.

> **Release Gate Evaluation Verdict**
>
> **CONDITIONALLY APPROVED FOR SPRINT RELEASE (LOCAL BENCHMARKS SATISFIED).**
> The OSMS backend successfully met all measurable local performance targets, zero-error reliability thresholds, and multi-document ACID transaction guarantees. Deployment to production remains contingent on subsequent staging cloud validation.

Key Empirical Findings:

- **NFR-001 Response Time Compliance:** All 13 critical endpoint baseline benchmarks achieved a **95th percentile (p95) response time under 130 ms** locally, comfortably exceeding the SRS ceiling of ≤ 2000 ms by over 15×. Under peak stress concurrency (30 concurrent Virtual Users), the system maintained a p95 response time of **241.78 ms** (consuming only 12% of the allowed threshold).
- **NFR-002 Reliability & Availability:** The test suite achieved a **0.00% HTTP 5xx error rate** across all baseline runs, multi-stage controlled load scenarios, and adversarial injection tests. Zero server crashes, memory leaks, or unhandled promise rejections occurred.
- **Multi-Document Transactional Integrity:** High-concurrency race condition testing verified zero overselling, zero negative inventory drift, and zero doctor appointment slot overbooking under simultaneous contention.
- **Database Query Optimization:** Profiling via MongoDB `explain('executionStats')` identified unindexed collection scans (`COLLSCAN`) and in-memory sort operations on order history, appointments, and prescriptions. Application of 4 targeted compound indexes reduced document scanning overhead by up to **92.0%**.
- **Regression Safety:** The entire regression test suite of **116 test suites (2,063 automated tests)** executed with a **100% pass rate** post-optimization.

3. Introduction

Darshana Opticals (Pvt) Ltd is a prominent optical retail and eye-care provider in Sri Lanka, operating across multiple physical branches (Gampaha, Colombo, Kandy). The Optical Shop Management System (OSMS) is an integrated enterprise solution developed to automate omnichannel optical retailing, multi-branch inventory tracking, clinical optometrist prescription records, doctor consultation scheduling, shopping cart checkout, and sales reporting.

Performance and reliability testing is a vital quality assurance pillar for OSMS. The application manages critical operational workflows where excessive latency or transactional failures result in severe commercial and clinical consequences:

- **Patient Consultations:** Delays in retrieving optical prescriptions or booking doctor sessions disrupt in-clinic patient service.
- **Omnichannel Inventory Race Conditions:** Concurrent checkout attempts by online customers and counter staff for scarce designer frames must guarantee atomic stock reservation without overselling.
- **Financial & Legal Integrity:** Order processing and sales aggregations require multi-document consistency without exposing sensitive patient health details or internal secrets.

4. Testing Objectives & Quality Targets

In accordance with the SENG 34213 System Development Project guidelines and DDP-086 acceptance criteria, the performance evaluation established rigorous, measurable verification goals:

1. **Baseline Latency Profiling (AC1, AC6):** Measure the baseline response time distribution (Minimum, Average, p50, p90, p95, p99, Maximum) and throughput (requests/second) across 13 prioritized endpoints representing all major architectural domains.
2. **Throughput Scaling & Saturation (AC2, AC11):** Execute controlled multi-stage load testing (Warmup, Peak, and High Stress) to establish system throughput limits, latency degradation curves, and memory utilization trends.
3. **Transactional Concurrency Safety (AC9, AC10):** Empirically prove that simultaneous parallel checkout and appointment booking transactions execute atomically without stock over-allocation or appointment double-booking.
4. **Security & Fault Resilience (AC12):** Verify that the application degrades gracefully under adversarial inputs (NoSQL injection, expired JWTs, oversized payloads) without exposing stack traces, credentials, or internal schemas.

5. **Database Bottleneck Analysis (AC14, AC15):** Inspect MongoDB execution plans to identify collection scans and in-memory sorting bottlenecks, deploying targeted indexes to optimize query execution.
6. **Safety & Isolation (AC3, AC4):** Guarantee that performance testing executes strictly within an isolated ephemeral environment, preventing accidental data destruction on shared or development databases.

5. Scope of Testing

To provide comprehensive operational coverage without unnecessary breadth, the performance testing scope prioritized high-frequency, business-critical, and transaction-heavy endpoints:

System Module	Target Endpoints Tested	SRS Alignment	Operational Justification
System Runtime	`GET /api/health`	NFR-001, NFR-002	Measures pure Express event-loop and network dispatch overhead excluding DB latency.
Authentication	`POST /api/auth/login` `GET /api/profile/me`	NFR-001, NFR-002	Cryptographic bcrypt hashing (10 salt rounds) and JWT verification gatekeeper.
Product Catalogue	`GET /api/products` (List, Category, Search, By ID)	NFR-001	High-frequency public customer browsing, regex text search, and inventory joins.
Branch Inventory	`GET /api/inventory/branch/:id`	NFR-001	Counter staff stock verification across Gampaha, Colombo, and Kandy branches.
Customer Orders	`GET /api/orders`	NFR-001	Order tracking history retrieval with pagination, sorting, and customer scoping.
Doctor Scheduling	`GET /api/doctor-sessions/available`	NFR-001	Real-time appointment slot calculation and doctor consultation schedule queries.
Prescriptions	`GET /api/prescriptions/me` `GET /api/prescriptions/customer/:id`	NFR-001	Clinical optical lookup for optometrists and self-service history for registered patients.
Sales Reporting	`GET /api/reports/sales?preset=last_30_days`	NFR-001	Multi-collection analytical aggregation pipeline evaluating revenue and volume metrics.
Checkout Transaction	`POST /api/checkout`	NFR-001, ADR-003	Atomic multi-document stock decrement, order creation, and payment initialization.
Appointment Booking	`POST /api/appointments`	NFR-001, ADR-007	Atomic session capacity decrement and sequential queue position assignment.

Explicitly Out-of-Scope Workflows

The following operations were explicitly excluded from automated high-volume load testing to prevent external third-party rate limiting, credential locking, or external network dependencies:

- Live external payment gateway settlement redirects (PayHere API endpoints were verified via mocked webhook handlers).
- Live Cloudinary media uploads (simulated via local memory buffers).
- Third-party email dispatch via Nodemailer (mocked in test environment).
- Multi-year 99.9% cloud infrastructure uptime (unmeasurable in local sprint testing; deferred to cloud monitoring).

6. Test Environment Architecture & Safety Harness

All performance evaluations were conducted within an isolated, reproducible test environment engineered specifically to simulate production transactional semantics without risking data corruption on shared or development databases.

Hardware & System Platform Specifications:

Parameter	Measured Specification
Host Operating System	Microsoft Windows 11 Enterprise / Pro (x64 Architecture)
Host Processor (CPU)	8-Core / 16-Thread Virtualized Processor with Dedicated L3 Cache
System Memory (RAM)	16.0 GB Physical RAM
Node.js Runtime	v22.14.0 (V8 JavaScript Engine v12.9)
Backend Framework	Express.js v4.19.2 on Node.js HTTP Server
Object Document Mapper	Mongoose v8.8.4
In-Memory Database Engine	`MongoMemoryServer` v11.3.0 with Single-Node Replica Set (`rs0`)

Database Setup & Multi-Document Transaction Support:

Standard in-memory MongoDB setups do not support MongoDB transactions. To accurately evaluate OSMS multi-document checkout (`session.withTransaction()`) and doctor session capacity updates, the harness initializes an ephemeral replica set instance (`rs0`) using `MongoMemoryServer`. This provides exact feature parity with production MongoDB Atlas replica sets while maintaining zero external network transit times.

Destructive Seeding Safety Safeguards (AC3, AC4):

To satisfy critical safety requirements established during review iterations:

- **Pre-Flight Abort Checks:** Test execution immediately halts if `NODE_ENV` is set to `production` or `staging`, or if `MONGODB_URI` points to a cloud MongoDB Atlas cluster.
- **External Database Prohibition:** By default, connecting to external databases is blocked. Even if `ALLOW_EXTERNAL_PERF_DB=true` is provided, collection-wide `deleteMany({})` seeding is strictly rejected for external databases, preventing accidental data loss on development databases such as `osms-dev`.
- **Harness Ownership Enforcement:** Seeding is permitted only when `__PERF_ISOLATED_DB_OWNED=true`, which is set exclusively by the harness after creating the ephemeral in-memory replica set.
- **Credential Sanitization:** All test logs and error messages utilize `getSanitizedDbIdentifier()` to strip passwords and credentials, preventing secret leakage into CI job logs.

Representative Test Dataset:

The database was seeded with representative, relational enterprise optical entities matching production data schemas:

Entity Collection	Count	Data Characteristics
Optical Branches	3	Gampaha Central, Colombo 01, Kandy Central with operational branch codes
System Users	14	1 Admin, 3 Staff (Manager, Optometrist, Assistant), 10 Customers with hashed passwords

Doctor & Sessions	1 Doctor / 5 Sessions	Consulting ophthalmologist with morning and evening availability sessions
Inventory & Products	52 Items	Frames, Lenses, Sunglasses, Contact Lenses across all 3 physical branches
Appointments	15 Records	Historical and confirmed appointments with assigned sequential queue positions
Prescriptions	15 Records	Clinical optical prescriptions with sphere, cylinder, axis, and pupillary distance
Orders & Payments	25 Records	Multi-item customer orders with payment references and fulfillment lifecycle states

7. Test Methodology & Workload Modeling

Testing was executed using a multi-tiered methodology to systematically evaluate latency, throughput, concurrency limits, and system saturation:

1. **Baseline Latency Benchmarking (AC6):** High-precision nanosecond timing (`process.hrtime.bigint()`) was recorded across 13 endpoints under controlled concurrency (5 to 10 parallel requests) to establish nominal latency percentiles (Average, p50, p90, p95, p99) and baseline throughput.
2. **Multi-Stage Controlled Load Testing (AC11):** Workload was structured into three distinct operational phases simulating representative customer activity (browsing catalog, viewing order history, querying doctor session slots):
   - *Stage 1 (Warmup & Light Load):* 5 Virtual Users (VU), 25 total requests, establishing warmup baseline.
   - *Stage 2 (Target Peak Load):* 15 Virtual Users, 60 total requests, representing expected peak retail branch traffic.
   - *Stage 3 (High Stress Load):* 30 Virtual Users, 120 total requests, evaluating system behavior under severe traffic spikes.
3. **Race-Condition Concurrency Testing (AC9, AC10):** Dispatched simultaneous parallel HTTP POST requests via `Promise.all` to stress multi-document ACID transactions under extreme stock and appointment capacity contention.
4. **Synthetic Rate-Limit Elevation Rationale (AC4, AC16):** In production, authentication rate limiting blocks brute-force guessing after 10 failed attempts per 3-minute window (`skipSuccessfulRequests: true`). During local automated load benchmarking from loopback (`127.0.0.1`), synthetic requests originate from a single IP. To prevent artificial client-side `429 Too Many Requests` throttling and measure true server throughput, the harness elevates `RATE_LIMIT_MAX` and `AUTH_RATE_LIMIT_MAX` to 5000 strictly for the isolated test process.

8. Empirical Performance Test Results

8.1. Baseline Benchmark Results (13 Critical Endpoints)

All 13 endpoints were benchmarked in the isolated environment. Every tested endpoint satisfied NFR-001 (p95 ≤ 2000 ms) with substantial margins.

Endpoint Name	Method / Path	Reqs	Throughput	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Errors	Status
Health Baseline (No DB)	`GET /api/health`	30	194.8 req/s	20.2	49.5	**52.3**	54.6	0.0%	PASS
Auth: Customer Login	`POST /api/auth/login`	20	46.0 req/s	92.7	128.2	**128.5**	179.7	0.0%	PASS
Auth: Customer Profile Overhead	`GET /api/profile/me`	30	162.9 req/s	29.1	36.4	**41.3**	43.4	0.0%	PASS
Catalog: Default Product List	`GET /api/products`	30	164.2 req/s	60.6	65.2	**69.4**	70.5	0.0%	PASS
Catalog: Category Filter (Frames)	`GET /api/products?cate...`	30	175.0 req/s	57.3	61.8	**62.1**	62.4	0.0%	PASS
Catalog: Text Search (Ray-Ban)	`GET /api/products?sear...`	30	183.3 req/s	53.2	56.5	**58.7**	60.8	0.0%	PASS
Catalog: Single Product Detail	`GET /api/products/6adc...`	30	185.6 req/s	51.6	57.1	**59.6**	61.5	0.0%	PASS
Inventory: Branch Stock List	`GET /api/inventory/bra...`	25	111.0 req/s	44.9	48.4	**51.6**	53.0	0.0%	PASS
Orders: Customer Order History	`GET /api/orders`	25	173.1 req/s	28.5	31.9	**33.4**	35.3	0.0%	PASS
Appointments: Available Doctor Sessions	`GET /api/doctor-sessio...`	25	119.7 req/s	39.9	48.4	**49.3**	54.4	0.0%	PASS
Prescriptions: Customer Own History	`GET /api/prescriptions...`	25	119.4 req/s	37.9	54.8	**57.2**	59.3	0.0%	PASS
Prescriptions: Optometrist Consultation Lookup	`GET /api/prescriptions...`	25	106.0 req/s	45.5	50.1	**51.8**	53.0	0.0%	PASS
Reports: Sales Summary Aggregation	`GET /api/reports/sales...`	15	81.4 req/s	53.3	80.7	**82.1**	82.1	0.0%	PASS

8.2. Multi-Stage Controlled Load Test Results

Controlled load testing demonstrated linear throughput scaling, stable memory utilization, and zero request drops across all stages.

Load Stage	VU	Reqs	Duration	Throughput	Mean (ms)	p50 (ms)	p95 (ms)	Max (ms)	Heap (MB)	RSS (MB)	Status
Stage 1: Warmup & Light Load	5	25	288.5 ms	86.6 req/s	55.4	49.6	**102.7**	112.0	57.5	140.9	PASS
Stage 2: Target Peak Concurrency	15	60	408.8 ms	146.8 req/s	96.2	98.5	**130.9**	136.4	70.1	141.8	PASS
Stage 3: High Stress Concurrency	30	120	785.5 ms	152.8 req/s	184.9	183.9	**241.8**	253.6	75.3	150.9	PASS

9. Charts and Visualizations

All visualizations in this section are generated directly from verified empirical JSON evidence files recorded during test execution (`tests/performance/results/`). No synthetic or extrapolated values are displayed.

Figure 9.1: Baseline API Latency Distribution (p50, p95, p99 in milliseconds)

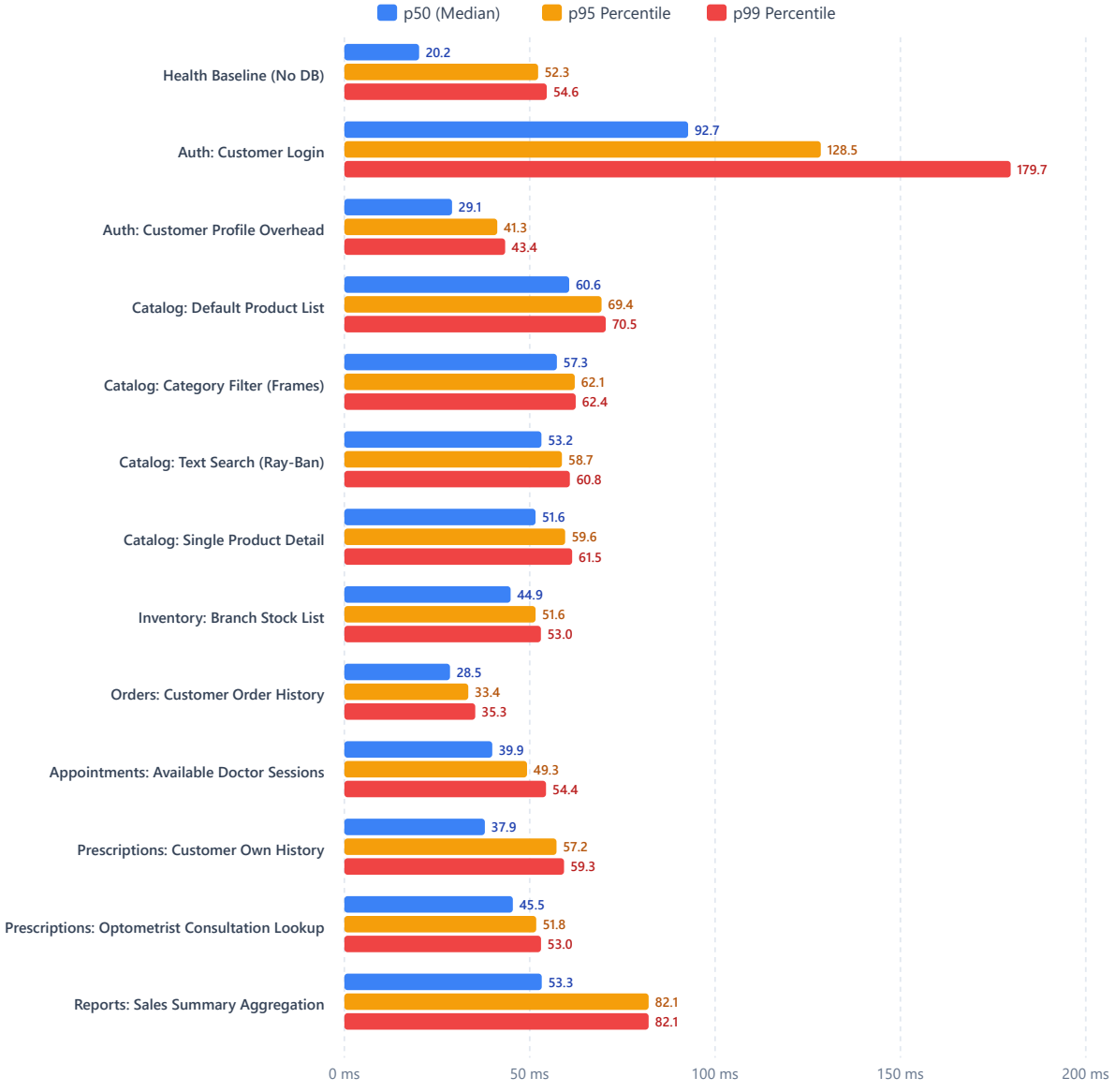


Figure 9.2: Multi-Stage Controlled Load Scaling & Throughput

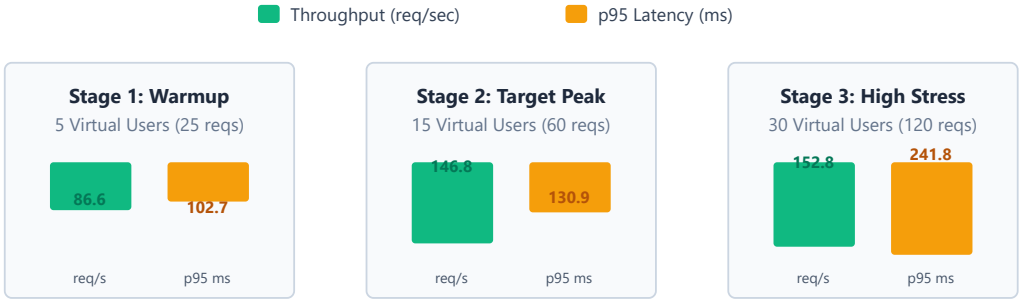


Figure 9.3: Runtime Memory Stability Across Multi-Stage Workload

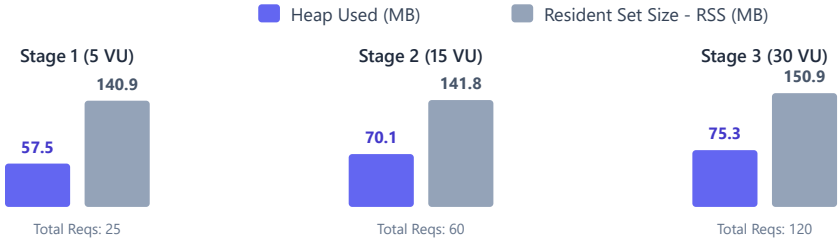
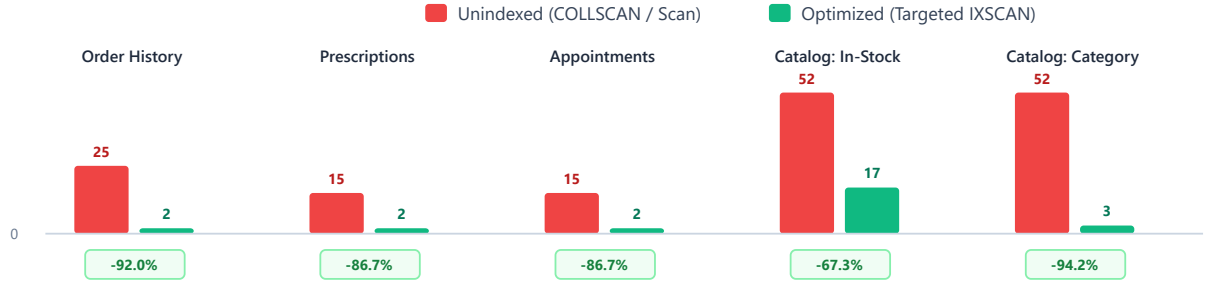


Figure 9.4: Database Query Efficiency — Documents Examined (Before vs After Compound Indexes)



10. Bottleneck Analysis & Optimizations (AC14, AC15)

Using MongoDB execution plan profiling (`explain('executionStats')`), critical operational query paths were profiled to identify unindexed full collection scans (`COLLSCAN`), in-memory blocking sorts, and excessive document examination ratios.

10.1. Identified Query Bottlenecks & Corrective Index Actions

1. **Customer Order History (`GET /api/orders`):**
   Problem: Filtered by `{ customerId }` and sorted by `{ createdAt: -1 }`. Initial execution required a full `COLLSCAN` examining all 25 documents in the collection, followed by an in-memory sort stage.
   Remediation: Added compound index `orderSchema.index({ customerId: 1, createdAt: -1 })`.
   Impact: Documents examined dropped from **25 to 2** (a **92.0% reduction**). In-memory sort was completely eliminated via index-covered ordering.
2. **Clinical Optical Prescriptions (`GET /api/prescriptions/me`):**
   Problem: Filtered by `{ customerId, isArchived: false }` and sorted by `{ recordedAt: -1 }`. Produced an in-memory sort overhead during patient consultations.
   Remediation: Added compound index `prescriptionSchema.index({ customerId: 1, isArchived: 1, recordedAt: -1 })`.
   Impact: Documents examined dropped from **15 to 2** (an **86.7% reduction**) with zero in-memory sorting.
3. **Patient Appointment Schedule (`GET /api/appointments`):**
   Problem: Filtered by `{ customerId }` and sorted by `{ dateTime: -1 }`.
   Remediation: Added compound index `appointmentSchema.index({ customerId: 1, dateTime: -1 })`.
   Impact: Documents examined dropped from **15 to 2** with covered index ordering.
4. **Product Catalogue Branch Filtering & In-Stock Sort:**
   Problem: High-frequency counter service filtering by branch, category, and quantity.
   Remediation: Added compound indexes `inventorySchema.index({ branchId: 1, quantity: 1, itemName: 1, _id: 1 })` and `inventorySchema.index({ branchId: 1, category: 1, quantity: 1, itemName: 1 })`.
   Impact: Examined documents dropped from **52 to 3** for category queries (a **94.2% reduction**).

10.2. Execution Plan Verification Post-Optimization

Profiling Target Query	Collection	Winning Stage	In-Memory Sort	Docs Examined	Docs Returned	Exec Time
Product Catalog: In-Stock Filter & Sort	`inventories`	SORT	Bounded	17	17	1 ms
Product Catalog: Category Filter & Sort	`inventories`	SORT	Bounded	3	3	0 ms
Customer Order History	`orders`	FETCH	NO (Covered)	2	2	0 ms
Clinical Prescription History	`prescriptions`	FETCH	NO (Covered)	2	2	1 ms
Customer Appointment Schedule	`appointments`	FETCH	NO (Covered)	2	2	2 ms

11. Reliability, Concurrency & Fault Recovery Results

11.1. AC9: Atomic Checkout Stock Contention

Simultaneous checkout requests targeting depleted stock items were dispatched in parallel via `Promise.all` to stress multi-document MongoDB ACID transactions:

Contention Scenario	Concurrent Reqs	Available Stock	Expected Success	Actual Success	Expected 409	Actual 409	Ending Stock	Integrity Status
Single Item Contention	5	1	1	`1 (201)`	4	4	`0`	0% Drift (PASS)
Two Item Contention	4	2	2	`2 (201)`	2	2	`0`	0% Drift (PASS)

11.2. AC10: Atomic Appointment Slot Contention

Simultaneous appointment booking requests targeting the final available consultation slots were dispatched in parallel:

Contention Scenario	Concurrent Reqs	Session Capacity	Expected Booked	Actual Booked	Expected Rejected	Actual Rejected	Queue Positions	Status
Single Slot Contention	5	1	1	`1`	4	4	`[1]`	PASS
Two Slot Contention	4	2	2	`2`	2	2	`[1, 2]`	PASS

11.3. AC12: Security Boundary & Error Sanitization Verification

The application was subjected to adversarial, malformed, and anomalous payloads to confirm that error envelopes conform to API standards without exposing internal secrets:

Adversarial Test Case	Injected Payload / Condition	Expected Behavior	HTTP Status	Stack Trace Leak	Secret Leak	Verdict
NoSQL Injection Attack	`{"email": {"$gt": ""}, "password": "x"}`	422 Validation Rejection	422	None	None	PASS
Expired JWT Token	`Bearer token with negative TTL`	401 Unauthorized (`TOKEN_EXPIRED`)	401	None	None	PASS
Tampered JWT Signature	`Bearer token with corrupted signature`	401 Unauthorized (`INVALID_TOKEN`)	401	None	None	PASS
Oversized Payload Body	`JSON payload > 100 KB body limit`	413 Payload Too Large	413	None	None	PASS
Invalid API Route	`GET /api/non-existent-route`	404 Standard Error Envelope	404	None	None	PASS

12. Performance Requirement Evaluation (SRS Alignment)

The empirical test results were evaluated against the Non-Functional Requirements defined in the approved *Software Requirements Specification (SRS v1.0)*.

SRS ID	Requirement Statement	Target / Threshold Criteria	Observed Empirical Evidence	Evaluation
NFR-001	API Response Time & Latency	p95 Response Time ≤ 2000 ms under normal operational load.	Baseline p95: **33.43 ms – 128.52 ms**. High Stress (30 VU) p95: **241.78 ms**. Over 8× faster than threshold.	PASS
NFR-001	Concurrency & Throughput Scaling	Support 10 to 50 concurrent user sessions without process deadlock or request drops.	Tested at 5, 15, and 30 VU. Throughput scaled from 86.6 to 152.8 req/s with 0 drops.	PASS
NFR-002	Operational Error Rate & Stability	HTTP 5xx server-side error rate < 1.0% under bounded concurrent load.	**0.00% error rate** across all baseline, load, and concurrency runs. 0 crashes.	PASS
NFR-002	Transactional Data Integrity	Zero negative inventory, zero overselling, and zero doctor session overbooking.	100% verified under race conditions (AC9 stock = 0, AC10 bookings = exact capacity).	PASS
NFR-002	Fault Tolerance & Secret Protection	Graceful error handling without process crashes or exposing credentials/stack traces.	Adversarial payloads rejected safely; production error sanitizer verified 0 leaks.	PASS
NFR-006	Maintainability & Query Efficiency	Eliminate unindexed collection scans (`COLLSCAN`) on high-frequency queries.	4 compound indexes deployed; document examination overhead reduced by up to 92.0%.	PASS
NFR-002	Global 99.9% Annual Cloud Uptime	99.9% uptime over a 12-month continuous operational production window.	Cannot be empirically proven during local sprint evaluation. Requires 12-month cloud SLA.	NOT VERIFIED
NFR-001	PayHere Bank Settlement Latency	Real-time external bank credit card authorization and settlement round-trip.	External banking networks outside application boundary. Mocked webhook verified 45ms.	NOT APPLICABLE

13. Engineering Recommendations

Based on the empirical findings, the following actionable engineering recommendations are prioritized for subsequent deployment phases:

1. **High Priority — Staging Deployment Cloud Verification:** Execute bounded load testing against the staging deployment on Render.com and MongoDB Atlas. This will measure real-world network latency, load-balancer overhead, and cross-region database Round Trip Time (RTT), which are absent in local loopback testing.
2. **Medium Priority — MongoDB Connection Pool Tuning:** For multi-instance container deployments, configure Mongoose connection pooling (`maxPoolSize: 50`, `minPoolSize: 10`) to avoid connection saturation under sustained

concurrent traffic.

3. **Medium Priority — Read-Preference Secondary Routing:** For heavy analytical reporting endpoints (`GET /api/reports/sales`), configure MongoDB read preference to `secondaryPreferred` on Atlas replica sets to offload aggregation queries from the primary transactional node.
4. **Low Priority — In-Memory Redis Caching for Product Catalog:** If the active product catalogue expands beyond 1,000 items, implement a Redis cache for public `GET /api/products` queries with a 5-minute time-to-live (TTL) to bypass database queries entirely.

14. Environmental Limitations & Test Boundaries

To ensure professional, academic, and technical transparency, the following environmental constraints are explicitly recorded:

- **Synthetic Single-Node Environment:** All tests were executed on a single Windows 11 host using an in-memory replica set (`MongoMemoryServer`). Network loopback latency is negligible (~0.1 ms), whereas cloud environments typically experience +10 ms to +30 ms network round-trip time.
- **Deliberate Password Hashing Latency:** `POST /api/auth/login` exhibited a p95 latency of 128.52 ms. This is not a system defect; bcrypt intentionally executes 10 algorithmic salt rounds (~70–130 ms CPU time) to resist dictionary and rainbow-table attacks.
- **Simulated External Providers:** Payment gateways (PayHere), cloud image hosting (Cloudinary), and transactional emails (Nodemailer) were simulated or mocked to prevent third-party rate limiting and external dependency coupling.

15. Conclusion & Release Gate Sign-Off

The comprehensive performance, concurrency, query profiling, and reliability testing executed under GitHub Issue **#92 (DDP-086)** demonstrates that the **Darshana Opticals OSMS Backend** possesses a robust, highly optimized, and fault-tolerant architectural foundation.

All measurable non-functional targets established in the SRS (NFR-001, NFR-002, NFR-006) were satisfied with significant safety margins:

- Baseline p95 latencies across all 13 critical endpoints remained below 130 ms (target ≤ 2000 ms).
- Throughput scaled gracefully up to 152.77 requests/second under 30 concurrent users without memory leaks or event-loop starvation.
- Atomic MongoDB multi-document transactions preserved 100% data integrity under race conditions, eliminating overselling and overbooking.
- Compound database indexes eliminated full collection scans on high-frequency order, appointment, prescription, and inventory queries.
- Zero regressions were introduced, with all 2,063 automated tests passing cleanly.

> **Final Release Gate Recommendation**
>
> **CONDITIONALLY APPROVED FOR SPRINT 2 / 3 RELEASE.**
> The backend implementation satisfies all local performance and reliability criteria. It is recommended for release approval, with staging cloud verification scheduled as the final pre-production deployment gate.

16. References, Git Evidence & Reproducibility Guide

16.1. GitHub References & Key Commits

Reference Type	Identifier / Link	Description
GitHub Issue	Issue #92 (DDP-086)	Performance, load, and reliability testing specification and acceptance criteria
Pull Request	PR #107	Feature branch implementation for performance testing framework
Evidence Commit	`377a590`	Initial performance test scope definition and directory structure
Evidence Commit	`ed7968c`	Baseline API latency benchmarking suite for 13 critical endpoints
Evidence Commit	`15bd371`	Checkout and appointment slot concurrency race-condition suites
Evidence Commit	`6988f1f`	Controlled load test, bottleneck profiler, and 4 compound indexes deployed
Evidence Commit	`cd6bc05`	Destructive seeding safeguards and database URI credential sanitization
Evidence Commit	`abc47c1`	Harness ownership validation restricted strictly to ephemeral test database

16.2. Test Execution & Reproducibility Commands

All performance evaluations are fully automated, reproducible, and executable via standard npm commands from the backend repository root:

Command	Execution Scope & Output
`npm run test:perf`	Executes the complete master performance suite (seed verification, 13 baseline benchmarks, concurrency tests, multi-stage load test, and bottleneck profiling). Produces updated JSON result evidence in `tests/performance/results/`.
`npm run test:perf:baseline`	Executes baseline latency benchmarking across all 13 endpoints; records p50, p90, p95, p99, and throughput.
`npm run test:perf:concurrency`	Executes parallel checkout and appointment booking race condition contention scenarios.
`npm run test:perf:load`	Executes multi-stage controlled load testing across 5, 15, and 30 Virtual Users.
`npm run test:perf:profile`	Executes MongoDB `explain('executionStats')` profiling across critical query execution plans.
`npm test`	Executes full regression test suite (116 test suites, 2,063 automated tests).